

Práctica PaaS: Despliegue de Servicio GenAI en Azure App Service (Rama Mínimo)

Este documento detalla la estructura, generación de imagen y flujo de comandos para realizar el despliegue del servicio mínimo en **Azure App Service (PaaS)** sobre Linux. El servicio consiste en un único contenedor que expone una API desarrollada en **FastAPI** que conecta con la API de **Gemini** para procesar prompts de usuario.

Componentes Azure Necesarios

Para llevar a cabo el despliegue de esta arquitectura en la nube, se requieren los siguientes componentes de **Microsoft Azure**:

- **Grupo de Recursos (Resource Group):** Contenedor lógico que agrupa todos los recursos relacionados con este despliegue para facilitar su administración y eliminación conjunta.
- **Azure Container Registry (ACR):** Registro privado de Docker en Azure donde se compilan, almacenan y gestionan las imágenes de los contenedores (**backend** y **frontend**).
- **Plan de App Service (App Service Plan):** Define los recursos físicos de computación y el sistema operativo (Linux con SKU B1) sobre los que correrá la aplicación.
- **Azure App Service (Web App for Containers):** Servicio PaaS que ejecuta la aplicación web a partir de la configuración multi-contenedor definida en el archivo **docker-compose.yml**.
- **Cuenta de Almacenamiento (Storage Account):** Servicio que aloja y gestiona las capacidades de almacenamiento en la nube requeridas para el proyecto.
- **Azure Files (File Share):** Recurso compartido de archivos montado como volumen persistente en la Web App (mapeado a **/app/data**) para salvaguardar el archivo de logs **history.txt** y asegurar su persistencia tras reinicios de los contenedores.

URLs del Servicio Desplegado en Azure

Nota: Todos los bloques de este documento comparten la misma Web App de Azure (**webapp-fastapi-gemini**), actualizándose iterativamente. La URL base es única y permanece constante a través de las distintas versiones desplegadas.

Endpoint	URL	Descripción
Generación de contenido	<code>https://webapp-fastapi-gemini.azurewebsites.net/generate?prompt=...</code>	Envía un prompt a Gemini y devuelve la respuesta
Historial de logs	<code>https://webapp-fastapi-gemini.azurewebsites.net/logs</code>	Devuelve el historial de comunicaciones Frontend→Backend (disponible desde Bloque 4)

Descripción de los Despliegues

La práctica implementa un servicio de Inteligencia Artificial Generativa (GenAI) basado en la API de **Google Gemini**, desplegado de forma iterativa en **Azure App Service (PaaS)** a través de cuatro bloques de complejidad creciente:

Bloque	Tipo	Descripción resumida
1	Contenedor único	Una sola imagen FastAPI que recibe prompts del usuario y los envía directamente a la API de Gemini. Sin persistencia, sin proxy. Se despliega en Azure App Service (Linux) usando Azure Container Registry (ACR).
2	Docker Compose — 2 contenedores	Se divide en un Frontend (gateway público en puerto 80) y un Backend (procesador Gemini en puerto 8080, sin exposición directa al exterior). El Frontend enruta las peticiones al Backend internamente. Despliegue multi-contenedor en Azure App Service.
3	Compose + Persistencia (backend)	Sobre el Bloque 2, el Backend guarda cada prompt en un archivo history.txt montado sobre Azure Files (File Share). El historial sobrevive a reinicios y reempliegues gracias al volumen persistente.
4	Compose + Persistencia (frontend) + Logs	El Frontend pasa a ser el responsable de registrar logs de comunicación (URL llamada, respuesta recibida) en history.txt . El volumen persistente se traslada al Frontend. Se añade el endpoint /logs para consultar el historial. Es la arquitectura final desplegada.

Todos los bloques utilizan imágenes propias construidas desde cero (no se usa DockerHub) y almacenadas en el **Azure Container Registry** planservicioia.azurecr.io.

Prerrequisitos del Entorno

Antes de comenzar a ejecutar los comandos de despliegue o pruebas locales, asegúrate de contar con los siguientes elementos instalados y configurados en tu equipo:

1. Docker Desktop

- **Uso:** Construcción, etiquetado y ejecución de contenedores locales, así como publicación de imágenes en el registro privado.
- **Instalación:** Descarga e instala [Docker Desktop para Windows](#).
- **Verificación:** Ejecuta el siguiente comando en PowerShell para verificar que el servicio está activo:

```
docker info
```

- **Inicio rápido (si no está activo):** Si Docker no se está ejecutando, puedes iniciarlo desde el menú de inicio de Windows o por PowerShell:

```
Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"
```

Espera a que el icono de Docker en la barra de tareas muestre "Docker Desktop is running" antes de continuar.

2. Azure CLI

- **Uso:** Administración, creación y configuración de toda la infraestructura PaaS (Azure App Service, Azure Container Registry, Azure Files).
- **Instalación:** Sigue la [guía de instalación de Azure CLI para Windows](#).
- **Verificación:** Ejecuta el siguiente comando en PowerShell para confirmar que está instalado correctamente:

```
az --version
```

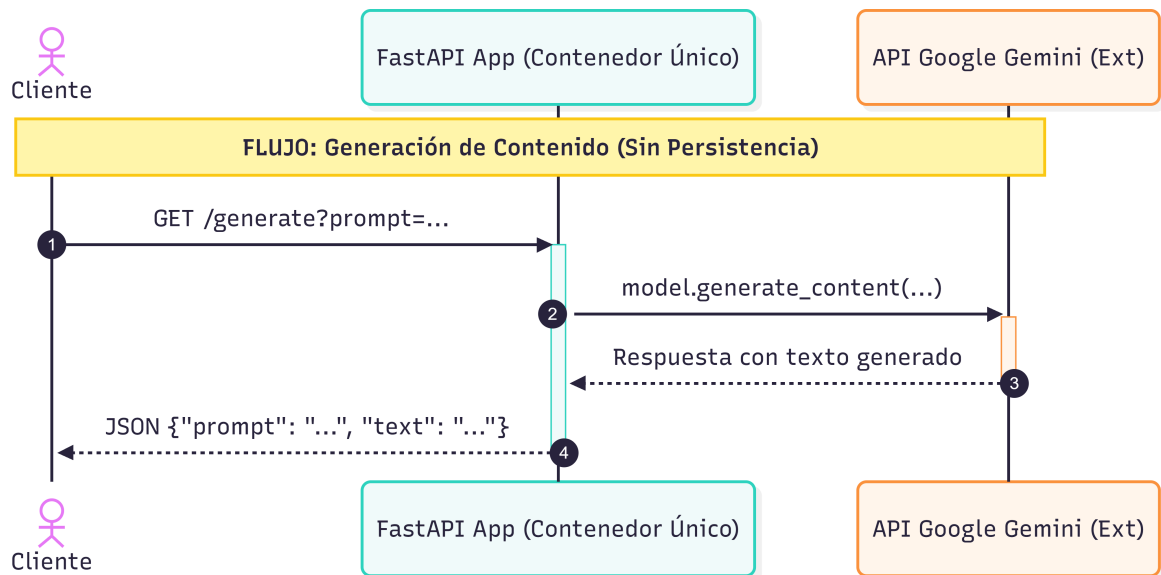
Bloque 1: Despliegue de un Contenedor

1.1. Descripción del Despliegue (Arquitectura Mínima)

El despliegue consiste en un servicio web gestionado en contenedor dentro de la plataforma **Azure App Service (Linux Web App)**.

Características del despliegue:

- **Sin Persistencia:** El contenedor funciona bajo el principio de infraestructura efímera (stateless). Las solicitudes se procesan en tiempo real y no se guarda historial de las consultas en archivos locales ni bases de datos.
- **Único Contenedor:** Toda la lógica reside en una sola aplicación de FastAPI, eliminando la necesidad de proxies o bases de datos adicionales.
- **Integración GenAI:** Se conecta directamente con la API oficial de Google Gemini usando el SDK de nueva generación (`google-generativeai`) y la clave de API `GEMINI_API_KEY` inyectada mediante variables de entorno en la nube.
- **Infraestructura PaaS:** Se emplea un plan de App Service en Linux, ideal para albergar APIs basadas en contenedores con escalado automático de recursos.



1.2. Definición del Contenedor (Dockerfile)

La imagen se construye a partir del siguiente **Dockerfile**, el cual optimiza el peso de la imagen mediante la versión **slim** de Python y aprovecha la caché de capas de Docker copiando primero las dependencias.

```

# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 8080

# Run the app with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
  
```

Código de la Aplicación (**app.py**):

El servicio expone un único endpoint **/generate** que interactúa de manera asíncrona con el modelo **gemini-2.5-flash**:

```
# app_fastapi_prompt.py
from dotenv import load_dotenv
from fastapi import FastAPI, Query, HTTPException
from google import genai
import os

# Carga variables de entorno locales (si existen)
load_dotenv()
client = genai.Client()

app = FastAPI()

@app.get("/generate")
async def generate_content(prompt: str = Query(..., description="The prompt to send to Gemini")):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt,
        )
        return {"prompt": prompt, "text": response.text}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

1.3. Pruebas y Validación en Local

Antes de proceder con el despliegue en la nube, se recomienda validar el correcto funcionamiento de la aplicación de manera local mediante Docker.

⚠ Prerequisito: Asegúrate de que **Docker Desktop** está arrancado antes de ejecutar cualquier comando **docker**. Puedes iniciarlo desde el menú de inicio de Windows o ejecutando **Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"** en PowerShell. Espera a que el icono de Docker en la barra de tareas muestre el mensaje *"Docker Desktop is running"* antes de continuar.

Ejecución mediante Docker local

Construye la imagen localmente y levanta el contenedor pasando la variable de entorno (para ejecución directa sin Docker, existe un archivo **.env** en la raíz del proyecto con la clave **GEMINI_API_KEY**):

```
# Leer la API Key desde el archivo .env (evita exponer la clave en el historial de comandos)
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })

# Construir la imagen local
docker build -t fastapi-gemini:local .

# Ejecutar el contenedor
```

```
docker run -d -p 8080:8080 --name fastapi-app-local -e
GEMINI_API_KEY=$env:GEMINI_API_KEY fastapi-gemini:local
```

Verificación local:

Realiza una llamada de prueba desde el navegador o consola al puerto local **8080**:

```
Invoke-WebRequest "http://localhost:8080/generate?prompt=Hello_Local_Testing" -
UseBasicParsing
```

i Nota sobre PowerShell: Se añade el parámetro `-UseBasicParsing` en todos los comandos `Invoke-WebRequest` de esta práctica para evitar que PowerShell intente inicializar el motor de Internet Explorer (lo cual suele lanzar una advertencia de seguridad interactiva y requerir confirmación del usuario para continuar).

```
PS C:\Users\danie\azure-assignment> docker build -t fastapi-gemini:local .
[+] Building 1.7s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 480B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.1s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 0.1s
=> [internal] load build context                                   0.0s
=> => transferring context: 63B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                      0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:2b73642c63558f8389f2c419bac2672ba67bb07bfea6363ec9a2bf181ba709ae 0.0s
=> => exporting config sha256:9f704624731f1017fa36cfc3ee12f8dc7217f09211cc0609afb9701c0db80fa 0.0s
=> => exporting attestation manifest sha256:ee96alab2682d0d827c667cedf16d50bce21b18c1384de01c5d1385f3eab8e5b 0.0s
=> => exporting manifest list sha256:ff20e99ae23746ecbfed1bcb5662a5299e3234f546a6a058f47bfbed48f5f9c 0.0s
=> => naming to docker.io/library/fastapi-gemini:local            0.0s
=> => unpacking to docker.io/library/fastapi-gemini:local         0.0s
PS C:\Users\danie\azure-assignment> docker run -d -p 8080:8080 --name fastapi-app-local -e GEMINI_API_KEY= fastapi-gemini:local
055273f9c45c893bd0f6b27fa42ebec9c0a0522c988ff15022b3b38e1a412531
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "http://localhost:8080/generate?prompt=Hello_Local_Testing"

Advertencia de seguridad: riesgo de ejecución de script
Invoke-WebRequest analiza el contenido de la página web. El código de script de la página web se puede ejecutar
cuando se analiza la página.
ACCIÓN RECOMENDADA:
Usa el modificador -UseBasicParsing para evitar la ejecución de código de script.

¿Quieres continuar?

[S] Sí [O] Sí a todo [N] No [T] No a todo [U] Suspender [?] Ayuda (el valor predeterminado es "N"): s

StatusCode      : 200
StatusDescription : OK
Content          : {"prompt":"Hello_Local_Testing","text":"Hello! How can I help you today?"}
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 74
                  Content-Type: application/json
                  Date: Mon, 01 Jun 2026 22:29:00 GMT
                  Server: uvicorn

Forms            : {"prompt":"Hello_Local_Testing","text":"Hello! How can I help you today?"}
Headers          : [{"Content-Length", 74}, [{"Content-Type", application/json}], [{"Date", Mon, 01 Jun 2026 22:29:00 GMT}], [{"Server", uvicorn}]]
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 74
```

1.4. Guía de Pasos Realizados (Despliegue en Azure)

A continuación se presenta la secuencia completa de comandos ejecutados en la consola para registrar la imagen de contenedor y aprovisionar la infraestructura en Azure.

Paso 1: Autenticación en Azure y definición de variables

Inicia sesión en tu suscripción de Azure mediante CLI y define las variables del entorno de trabajo:

```
# Autenticarse en Azure
az login

# Definir variables de entorno para los comandos
$RESOURCE_GROUP = "gruporecursosia"
$LOCATION = "spaincentral"
$ACR_NAME = "planservicioia"
$APP_SERVICE_PLAN = "plan-fastapi-gemini"
$WEB_APP_NAME = "webapp-fastapi-gemini"
$IMAGE_TAG = "planservicioia.azurecr.io/fastapi-gemini:v1"
```

Paso 2: Creación del Grupo de Recursos y del Azure Container Registry (ACR)

Crea un grupo de recursos donde se alojarán todos los componentes y aprovisiona el registro de contenedores privado:

```
# Crear el grupo de recursos
az group create --name $RESOURCE_GROUP --location $LOCATION
```

```
PS C:\Users\danie> $RESOURCE_GROUP = "gruporecursosia"
PS C:\Users\danie> $LOCATION = "spaincentral"
PS C:\Users\danie> $ACR_NAME = "planservicioia"
PS C:\Users\danie> $APP_SERVICE_PLAN = "plan-fastapi-gemini"
PS C:\Users\danie> $WEB_APP_NAME = "webapp-fastapi-gemini"
PS C:\Users\danie> $IMAGE_TAG = "planservicioia.azurecr.io/fastapi-gemini:v1"
PS C:\Users\danie> az group create --name $RESOURCE_GROUP --location $LOCATION
{
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia",
  "location": "spaincentral",
  "managedBy": null,
  "name": "gruporecursosia",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
PS C:\Users\danie> |
```

```
# Crear el Azure Container Registry (SKU básico para desarrollo/pruebas)
az acr create --name $ACR_NAME --resource-group $RESOURCE_GROUP --sku Basic
```

```
PS C:\Users\danie> az acr create --name $ACR_NAME --resource-group $RESOURCE_GROUP --sku Basic
{
  "adminUserEnabled": false,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-06-01T21:42:08.509823+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.ContainerRegistry/registries/planservicioia",
  "identity": null,
  "location": "spaincentral",
  "loginServer": "planservicioia.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "planservicioia",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2026-06-01T21:42:25.671511+00:00",
      "status": "disabled"
    },
    "softDeletePolicy": {
      "lastUpdatedTime": "2026-06-01T21:42:25.671544+00:00",
      "retentionDays": 7,
      "status": "disabled"
    },
    "trustPolicy": {
      "status": "disabled",
      "type": "Notary"
    }
  },
  "privateEndpointConnections": [],
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "gruporecursosia",
  "roleAssignmentMode": "LegacyRegistryPermissions",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  },
  "status": null,
  "systemData": {
    "createdAt": "2026-06-01T21:42:08.509823+00:00",
    "createdBy": "daniel.quintillan@udc.es",
    "createdByType": "User",
    "lastModifiedAt": "2026-06-01T21:42:08.509823+00:00",
    "lastModifiedBy": "daniel.quintillan@udc.es",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.ContainerRegistry/registries",
  "zoneRedundancy": "Disabled"
}
PS C:\Users\danie> |
```

Paso 3: Habilitar usuario administrador en ACR y autenticarse localmente

Habilita el acceso de administrador para obtener las credenciales de lectura/escritura de imágenes directamente desde Docker:

```
# Habilitar usuario admin del registro
az acr update -n $ACR_NAME --admin-enabled true
```



```
PS C:\Users\danie> az acr update -n $ACR_NAME --admin-enabled true
{
  "adminUserEnabled": true,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-06-01T21:42:08.509823+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.ContainerRegistry/registries/planservicioia",
  "identity": null,
  "location": "spaincentral",
  "loginServer": "planservicioia.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "planservicioia",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2026-06-01T21:42:25.671511+00:00",
      "status": "disabled"
    },
    "softDeletePolicy": {
      "lastUpdatedTime": "2026-06-01T21:42:25.671544+00:00",
      "retentionDays": 7,
      "status": "disabled"
    },
    "trustPolicy": {
      "status": "disabled",
      "type": "Notary"
    }
  },
  "privateEndpointConnections": [],
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "gruporecursosia",
  "roleAssignmentMode": "LegacyRegistryPermissions",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  },
  "status": null,
  "systemData": {
    "createdAt": "2026-06-01T21:42:08.509823+00:00",
    "createdBy": "daniel.quintillan@udc.es",
    "createdByType": "User",
    "lastModifiedAt": "2026-06-01T22:11:14.087718+00:00",
    "lastModifiedBy": "daniel.quintillan@udc.es",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.ContainerRegistry/registries",
  "zoneRedundancy": "Disabled"
}
PS C:\Users\danie> |
```

Obtener la contraseña de administrador

```
$ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)
```

Iniciar sesión local en Docker con las credenciales de ACR (usando password-stdin por seguridad)

```
echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --
password-stdin
```

```
PS C:\Users\danie> $ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query "passwords[0].value" -o tsv)
PS C:\Users\danie> docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password $ACR_PASSWORD
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
Login Succeeded
PS C:\Users\danie> |
```

Paso 4: Construcción y subida de la imagen Docker

Construye la imagen Docker localmente etiquetándola con la URI del registro de Azure, y súbela al repositorio:

```
# Construir la imagen usando el Dockerfile de la raíz
docker build -t $IMAGE_TAG .
```

```
PS C:\Users\danie\azure-assignment> docker build -t $IMAGE_TAG .
[+] Building 40.4s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 480B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.4s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 7.8s
=> => resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382 0.1s
=> => sha256:45006ceeeea9e2ca59a046b6f4ac9a212e4b27b3d8d6e7d348b03f0aaccdac99 250B / 250B      0.2s
=> => sha256:797d495f2c68eb4664df918d39078ca04612d7ef47c57b5c784f7f6eadd42bf5 14.37MB / 14.37MB 2.4s
=> => sha256:8649771fee179d7c2590c94f533aaa5fceb70e36e25dec83672fe836d99577c5 1.29MB / 1.29MB 1.0s
=> => sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 29.78MB / 29.78MB 3.8s
=> => extracting sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 1.9s
=> => extracting sha256:8649771fee179d7c2590c94f533aaa5fceb70e36e25dec83672fe836d99577c5 0.3s
=> => extracting sha256:797d495f2c68eb4664df918d39078ca04612d7ef47c57b5c784f7f6eadd42bf5 1.6s
=> => extracting sha256:45006ceeeea9e2ca59a046b6f4ac9a212e4b27b3d8d6e7d348b03f0aaccdac99 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 750B                                       0.0s
=> [2/5] WORKDIR /app                                             0.6s
=> [3/5] COPY requirements.txt .                                  0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt      23.1s
=> [5/5] COPY app.py .                                           0.1s
=> exporting to image                                             6.0s
=> => exporting layers                                                3.4s
=> => exporting manifest sha256:2b73642c63558f8389f2c419bac2672ba67bb07bfea6363ec9a2bf181ba709ae 0.0s
=> => exporting config sha256:9f704624731f1017fa36cfcf3ee12f8dc7217f09211cc0609afb9701c0db80fa 0.0s
=> => exporting attestation manifest sha256:12fa9a979a002e6b9ea70cc549c80bab673b95a44b6865decdd8d4e12b2198fd 0.0s
=> => exporting manifest list sha256:58291f09ae9cc90f11613e925afc7113657a4f339fe9a32fc7a8c612bb384253 0.0s
=> => naming to planservicioia.azurecr.io/fastapi-gemini:v1        0.0s
=> => unpacking to planservicioia.azurecr.io/fastapi-gemini:v1     2.3s
PS C:\Users\danie\azure-assignment>
```

```
# Subir la imagen a Azure Container Registry
docker push $IMAGE_TAG
```

```
PS C:\Users\danie\azure-assignment> docker push $IMAGE_TAG
The push refers to repository [planservicioia.azurecr.io/fastapi-gemini]
949a5aa2f50a: Pushed
08f1375a4fa3: Pushed
797d495f2c68: Pushed
e285e1798bd9: Pushed
5b4d6ff92fc4: Pushed
8649771fee17: Pushed
5dbf852422f1: Pushed
97ff604ed1b0: Pushed
45006ceeeea9: Pushed
v1: digest: sha256:58291f09ae9cc90f11613e925afc7113657a4f339fe9a32fc7a8c612bb384253 size: 856
PS C:\Users\danie\azure-assignment>
```

Paso 5: Creación del Plan de App Service

Aprovisiona un plan de hospedaje en Azure App Service configurado en Linux:

```
# Crear el Plan de App Service (SKU B1 de Linux como nivel básico de desarrollo)
az appservice plan create --name $APP_SERVICE_PLAN --resource-group
$RESOURCE_GROUP --sku B1 --is-linux
```

```
PS C:\Users\danie\azure-assignment> az appservice plan create --name $APP_SERVICE_PLAN --resource-group $RESOURCE_GROUP --sku B1 --is-linux
{
  "asyncScalingEnabled": false,
  "elasticScaleEnabled": false,
  "extendedLocation": null,
  "freeOfferExpirationTime": "2026-12-01T22:39:08.233333",
  "geoRegion": "Spain Central",
  "hostingEnvironmentProfile": null,
  "hyperV": false,
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.Web/serverfarms/plan-fastapi-gemini",
  "isSpot": false,
  "isXenon": false,
  "kind": "linux",
  "kubeEnvironmentProfile": null,
  "location": "spaincentral",
  "maximumElasticWorkerCount": 1,
  "maximumNumberOfWorkers": 0,
  "name": "plan-fastapi-gemini",
  "numberOfSites": 0,
  "numberOfWorkers": 1,
  "perSiteScaling": false,
  "provisioningState": "Succeeded",
  "reserved": true,
  "resourceGroup": "gruporecursosia",
  "sku": {
    "capabilities": null,
    "capacity": 1,
    "family": "B",
    "locations": null,
    "name": "B1",
    "size": "B1",
    "skuCapacity": null,
    "tier": "Basic"
  },
  "spotExpirationTime": null,
  "status": "Ready",
  "subscription": "884c87e5-7027-435d-905b-1722c9bcfddc",
  "tags": null,
  "targetWorkerCount": 0,
  "targetWorkerSizeId": 0,
  "type": "Microsoft.Web/serverfarms",
  "workerTierName": null,
  "zoneRedundant": false
}
```

Paso 6: Despliegue de la Web App desde el ACR

Aprovisiona la Web App de contenedores configurando la imagen previamente cargada en el registro de Azure:

```
# Crear la Web App con la imagen del contenedor de Azure
az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name
$WEB_APP_NAME --deployment-container-image-name $IMAGE_TAG
```

```
PS C:\Users\danie\azure-assignment> az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name $WEB_APP_NAME --deployment-container-image-name $IMAGE_TAG
Option '-deployment-container-image-name' has been deprecated and will be removed in a future release.
vnet_route_all_enabled is not a known attribute of class <class 'azure.mgmt.web.v2024_11_01.models._models_py3.Site'> and will be ignored
No credential was provided to access Azure Container Registry. Trying to look up...
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "1FE448438D76EB3CD89F2F8CDC94FD32281C423F2A99F8C844D6FA3298A1AE47",
  "dailyMemoryLimitQuota": 0,
  "daprConfig": null,
  "defaultHostName": "webapp-fastapi-gemini.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,
  "enabledHostNames": [
    "webapp-fastapi-gemini.azurewebsites.net",
    "webapp-fastapi-gemini.scm.azurewebsites.net"
  ],
  "endToEndEncryptionEnabled": false,
  "extendedLocation": null,
  "ftpPublishingUrl": "https://waws-prod-esc-003.ftp.azurewebsites.windows.net/site/wwwroot",
  "functionAppConfig": null,
  "hostNameSslStates": [
    {
      "certificateResourceId": null,
      "hostType": "Standard",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    },
    {
      "certificateResourceId": null,
      "hostType": "Repository",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.scm.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    }
  ],
  "hostNames": [

```

Paso 7: Configuración de Credenciales de Registro y Variables de Entorno

Configura la autenticación del App Service contra el ACR para que pueda descargar las actualizaciones de la imagen y define los secretos de la aplicación:

```
# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-Object { $_.ToString().Split("=",2)[1] })

# Vincular las credenciales del registro privado a la Web App (usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP --docker-custom-image-name $IMAGE_TAG --container-registry-url "https://$ACR_NAME.azurecr.io" --container-registry-user $ACR_NAME --container-registry-password $ACR_PASSWORD

# Definir la API Key de Gemini y forzar el puerto 8080 dentro del contenedor
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=8080 GEMINI_API_KEY=$GEMINI_API_KEY
```

```
PS C:\Users\danie\azure-assignment> az webapp config container set --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP --docker-custom-image-name $IMAGE_TAG --docker-registry-server-url "https://$SACK_NAME.azurecr.io" --docker-registry-server-password $SACK_PASSWORD
Option '--docker-custom-image-name' has been deprecated and will be removed in a future release. Use '--container-image-name' instead.
Option '--docker-registry-server-url' has been deprecated and will be removed in a future release. Use '--container-registry-url' instead.
Option '--docker-registry-server-user' has been deprecated and will be removed in a future release. Use '--container-registry-user' instead.
Option '--docker-registry-server-password' has been deprecated and will be removed in a future release. Use '--container-registry-password' instead.
{
  "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
  "slotSetting": false,
  "value": "false"
},
{
  "name": "DOCKER_REGISTRY_SERVER_URL",
  "slotSetting": false,
  "value": "https://plan.servicioia.azurecr.io"
},
{
  "name": "DOCKER_REGISTRY_SERVER_USERNAME",
  "slotSetting": false,
  "value": "plan.servicioia"
},
{
  "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
  "slotSetting": false,
  "value": null
},
{
  "name": "DOCKER_CUSTOM_IMAGE_NAME",
  "value": "DOCKER|plan.servicioia.azurecr.io/fastapi-gemini:v1"
}
}
PS C:\Users\danie\azure-assignment> az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=8080 GEMINI_API_KEY=[REDACTED]
{
  "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
  "slotSetting": false,
  "value": null
},
{
  "name": "DOCKER_REGISTRY_SERVER_URL",
  "slotSetting": false,
  "value": null
},
{
  "name": "DOCKER_REGISTRY_SERVER_USERNAME",
  "slotSetting": false,
  "value": null
},
{
  "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
  "slotSetting": false,
  "value": null
},
{
  "name": "WEBSITES_PORT",
  "slotSetting": false,
  "value": null
},
{
  "name": "GEMINI_API_KEY",
  "slotSetting": false,
  "value": null
}
}
PS C:\Users\danie\azure-assignment> |
```

1.5. Verificación del Funcionamiento en Azure

Una vez que el despliegue haya completado su inicio en Azure, puedes probar el endpoint de generación ejecutando la siguiente llamada HTTP desde la consola:

```
# Realizar petición de prueba a la API desplegada en Azure
# Opción A: usando la variable (requiere haberla definido antes en la sesión)
Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?
prompt=Explain_PaaS_in_one_sentence" -UseBasicParsing

# Opción B: con la URL literal (funciona en cualquier sesión)
Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?
prompt=Explain_PaaS_in_one_sentence" -UseBasicParsing
```

Formato esperado de respuesta:

```
{
  "prompt": "Explain_PaaS_in_one_sentence",
  "text": "Platform as a Service (PaaS) is a cloud computing model where a third-party provider delivers hardware and software tools—usually those needed for application development—to users over the internet."
}
```

```
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Explain_PaaS_in_one_sentence"
Advertencia de seguridad: riesgo de ejecución de script
Invoke-WebRequest analiza el contenido de la página web. El código de script de la página web se puede ejecutar cuando se analiza la página.
ACCIÓN RECOMENDADA:
Usa el modificador -UseBasicParsing para evitar la ejecución de código de script.
¿Quieres continuar?
[S] Sí [O] No [T] No a todo [U] Suspender [?] Ayuda (el valor predeterminado es "N"): s

StatusCode      : 200
StatusDescription: OK
Content          : [{"prompt":"Explain_PaaS_in_one_sentence","text":"PaaS provides a cloud-based platform that enables developers to build, run, and manage applications without having to provision or manage the underlyin...
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 219
                  Content-Type: application/json
                  Date: Mon, 01 Jun 2026 22:49:47 GMT
                  Set-Cookie: ARRAffinity=279e2686dd6985a92f0e5544b67f8addb363f77898a222be35d5034f8df4ba;Path=/
Forms            : {}
Headers          : [{"Content-Length", 219}, {"Content-Type", application/json}, {"Date", Mon, 01 Jun 2026 22:49:47 GMT}, {"Set-Cookie", ARRAffinity=279e2686dd6985a92f0e5544b67f8addb363f77898a222be35d5034f8df4ba;Path=/;HttpOnly;Secure;Domain=web...
                  tes.net, ARRAffinitySameSite=279e2686dd6985a92f0e5544b67f8addb363f77898a222be35d5034f8df4ba;Path=/;HttpOnly;SameSite=None;Secure;Domain=webapp-fastapi-gemini.azurewebsites.net}...]
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mhtml:HTMLDocumentClass
RawContentLength : 219
```

1.5.1. Limpieza de Recursos y Control de Gastos

Para evitar que los recursos aprovisionados sigan consumiendo créditos de estudiante o facturando cargos adicionales, se recomienda gestionar el ciclo de vida de los servicios mediante una de las siguientes opciones:

Si deseas conservar la configuración de la Web App pero detener la ejecución del contenedor:

```
# Detener la Web App
az webapp stop --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

# Volver a iniciar el servicio cuando sea necesario
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

Bloque 2: Despliegue Docker Compose con Dos Contenedores

En esta segunda iteración, la aplicación se estructura en una arquitectura de múltiples contenedores coordinados por Docker Compose. Esta arquitectura divide el monolito en:

1. **Frontend API (Gateway):** Un contenedor FastAPI en el puerto 80 que recibe todas las peticiones externas y las redirige hacia la capa interna.
2. **Backend API:** El contenedor FastAPI original en el puerto 8080 que se comunica exclusivamente con la API de Gemini, protegido del exterior.

2.1. Estructura de Docker Compose

En la raíz del proyecto se ha introducido un archivo `docker-compose.yml` que orquesta ambos servicios. Este archivo ya cuenta con el nombre y URL de su ACR resueltos directamente en las imágenes, por lo que no requiere sustitución dinámica de variables:

```
version: '3.8'

services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v1
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"
```

```
frontend:
  build: ./frontend
  image: planservicioia.azurecr.io/fastapi-frontend:v1
  environment:
    - BACKEND_URL=http://backend:8080
  ports:
    - "80:80"
  depends_on:
    - backend
```

2.2. Pruebas Locales (Docker Compose)

En lugar de construir una sola imagen, utiliza compose para levantar el clúster entero simultáneamente:

 **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → "Docker Desktop is running").

```
# Leer la API Key desde el archivo .env
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })
docker-compose up --build -d
```

Verifique apuntando al puerto 80 (el cual ahora es gestionado por el Gateway frontend):

```
Invoke-WebRequest "http://localhost:80/generate?prompt=Prueba_Compose"
```

```

PS C:\Users\danie\azure-assignment> docker-compose up --build -d
>
time="2026-06-02T23:39:03+02:00" level=warning msg="C:\\Users\\danie\\azure-assignment\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 2.1s (28/28) FINISHED
-> [internal] load local bake definitions                                0.0s
-> -> reading from stdin 1.88kB                                         0.0s
-> [backend internal] load build definition from Dockerfile             0.0s
-> -> transferring dockerfile: 560B                                      0.0s
-> [frontend internal] load build definition from Dockerfile           0.0s
-> -> transferring dockerfile: 487B                                      0.0s
-> [frontend internal] load metadata for docker.io/library/python:3.11-slim 0.9s
-> [frontend internal] load .dockerignore                               0.0s
-> -> transferring context: 2B                                           0.0s
-> [backend internal] load .dockerignore                                0.0s
-> -> transferring context: 2B                                           0.0s
-> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833c22893cb848908979487a9fc8e6c38295747e48ac9 0.0s
-> -> resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833c22893cb848908979487a9fc8e6c38295747e48ac9 0.0s
-> [frontend internal] load build context                               0.0s
-> -> transferring context: 63B                                           0.0s
-> [backend internal] load build context                               0.0s
-> -> transferring context: 63B                                           0.0s
-> CACHED [backend 2/5] WORKDIR /app                                    0.0s
-> CACHED [frontend 3/5] COPY requirements.txt .                        0.0s
-> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [frontend 5/5] COPY app.py .                                  0.0s
-> CACHED [backend 3/5] COPY requirements.txt .                        0.0s
-> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [backend 5/5] COPY app.py .                                  0.0s
-> [frontend] exporting to image                                        0.2s
-> -> exporting layers                                                    0.0s
-> -> exporting manifest sha256:b0c1f9f6ca0fc5b6b747af4c20f4c1aceb5c77d671654618b42b2834a38b857 0.0s
-> -> exporting config sha256:8f396658f32778dc8d8dbcb88b8a1a7caeb6cb449616898c067d4d18e6953120 0.0s
-> -> exporting attestation manifest sha256:b0d807f90734aa157b2a2bb1ca3867f027e91bb373435a6a164b8cef9c9fecb 0.1s
-> -> exporting manifest list sha256:9567ab082470844df0e430d7d7b7c6a8bc5d83e72a28ba175cbb5db035a3a148 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-frontend:v1          0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-frontend:v1       0.0s
-> [backend] exporting to image                                        0.2s
-> -> exporting layers                                                    0.0s
-> -> exporting manifest sha256:71a44f05f68d4f28c956f5f0308928574601616b07b72c79ba4f80bf5781c5d 0.0s
-> -> exporting config sha256:bfc579c78ff041a871976a82f0f478e4bb477af63a37f0bb350fcb1891927d 0.0s
-> -> exporting attestation manifest sha256:3464b41df02a01bb65326f0ec71d9374f53bfc4a658720c24fcbca974ee0d4dd 0.1s
-> -> exporting manifest list sha256:d1523252c6d80e59ea17a53f822e11b282ca88bdc89273a43a53d678a929e4ef 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-backend:v1          0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-backend:v1       0.0s
-> [frontend] resolving provenance for metadata file                   0.0s
-> [backend] resolving provenance for metadata file                   0.0s
[*] Running 2/2
✓ planservicioia.azurecr.io/fastapi-frontend:v1 Built                0.0s
[*] Running 4/4ia.azurecr.io/fastapi-backend:v1 Built                0.0s
✓ planservicioia.azurecr.io/fastapi-frontend:v1 Built                0.0s
✓ planservicioia.azurecr.io/fastapi-backend:v1 Built                0.0s
✓ Container azure-assignment-backend-1 Started                       2.0s
✓ Container azure-assignment-frontend-1 Started                      1.4s
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "http://localhost:80/generate?prompt=Prueba_Compose"

StatusCode      : 200
StatusDescription : OK
Content         : {"prompt":"Prueba_Compose","text":"¡Claro! Parece que quieres una introducción o una \"prueba\" de Jetpack Compose. ¡Es una excelente elección! Jetpack Compose es el kit de herramientas moderno de Android."}
RawContent      : HTTP/1.1 200 OK
                  Date: Tue, 02 Jun 2026 21:39:18 GMT
                  Server: uvicorn
                  Content-Length: 17823
                  Content-Type: application/json

                  {"prompt":"Prueba_Compose","text":"¡Claro! Parece que quieres una introd...
Headers         : [{"Date", System.String[]}, {"Server", System.String[]}, {"Content-Length", System.String[]}, {"Content-Type", System.String[]}]
Images          : {}
InputFields     : {}
Links           : {}
RawContentLength : 17823
RelationLink    : {}

```

2.3. Despliegue en Azure App Service Multi-Container

Para desplegar la aplicación en Azure usando esta topología, los comandos a ejecutar cambian ligeramente. Si has iniciado una nueva sesión de terminal, redefine primero las variables necesarias:

```

$RESOURCE_GROUP = "gruporecursosia"
$LOCATION = "spaincentral"
$ACR_NAME = "planservicioia"
$APP_SERVICE_PLAN = "plan-fastapi-gemini"
$WEB_APP_NAME = "webapp-fastapi-gemini"
$ACR_PASSWORD = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)

```

A. Construcción y subida de ambas imágenes al ACR


```
# Exportamos el nombre del ACR para los tags del Compose
$env:ACR_NAME=$ACR_NAME

# Autenticarse en el ACR antes de hacer push
docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password $ACR_PASSWORD

# Construimos y subimos las imágenes
docker-compose build
docker-compose push
```

```
C:\Users\danie\azure-assignment> semv\ACR_NAME=SACRIAME
PS C:\Users\danie\azure-assignment> docker-compose build
time="2026-06-03T18:17:16+02:00" level=warning msg="C:\\Users\\danie\\azure-assignment\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 6.3s (28/28) FINISHED
=> [internal] load local bake definitions      0.1s
=> => reading from stdin 1.88kB              0.1s
=> [frontend internal] load build definition from Dockerfile 0.1s
=> => transferring dockerFile: 487B           0.0s
=> [backend internal] load build definition from Dockerfile 0.1s
=> => transferring dockerFile: 580B           0.0s
=> [frontend internal] load metadata for docker.io/library/python:3.11-slim 1.7s
=> [frontend internal] load .dockerignore      0.0s
=> => transferring context: 2B                0.0s
=> [backend internal] load .dockerignore       0.1s
=> => transferring context: 2B                0.0s
=> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0966bc4e91546ae033e22093cb840908b79487 0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:a3ab0966bc4e91546ae033e22093cb840908b79487a9fcbe6e 0.1s
=> [frontend internal] load build context     0.0s
=> => transferring context: 63B               0.0s
=> [backend internal] load build context      0.0s
=> => transferring context: 63B               0.0s
=> CACHED [backend 2/5] WORKDIR /app          0.0s
=> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [frontend 5/5] COPY app.py .        0.0s
=> CACHED [backend 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [backend 5/5] COPY app.py .        0.0s
=> => transferring context: 63B               0.0s
=> CACHED [backend 2/5] WORKDIR /app          0.0s
=> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [frontend 5/5] COPY app.py .        0.0s
=> CACHED [backend 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [backend 5/5] COPY app.py .        0.0s
=> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [frontend 5/5] COPY app.py .        0.0s
=> CACHED [backend 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [backend 5/5] COPY app.py .        0.0s
=> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [frontend 5/5] COPY app.py .        0.0s
=> CACHED [backend 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [backend 5/5] COPY app.py .        0.0s
planervicioia.azurecr.io/fastapi-frontend:v1 Built 0.0s
PS C:\Users\danie\azure-assignment> docker compose push
time="2026-06-03T18:18:45+02:00" level=warning msg="C:\\Users\\danie\\azure-assignment\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Pushing 16/18
Pushing planervicioia.azurecr.io/fastapi-frontend:v1: ba642724926a Pushing [=====][+] Pushing 16/18=====> 1.219Kb/1.219Kb.5s v1: 05f299aa7d3c Waiting 0.4s
Pushing planervicioia.azurecr.io/fastapi-frontend:v1: ba642724926a Pushing [=====][+] Pushing 18/18=====> 1.219Kb/1.219Kb.6s v1: 7976495f2c68 Waiting 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: ba642724926a Pushed 0.7s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 05f299aa7d3c Pushed 0.6s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 45086ceeeea9 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 7976495f2c68 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 411cd8e53846 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 9b4d6ff92fc4 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 5dbf852422f1 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: c28e517908b9 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 7976495f2c68 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 5b4d6ff92fc4 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 45086ceeeea9 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: b272f22bc825 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 8649771fee17 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 3e75f8258abc Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-backend:v1: 97ffe04ed1b0 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 2d4e344ad999 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 8649771fee17 Layer already exists 0.4s
✓ Pushing planervicioia.azurecr.io/fastapi-frontend:v1: 5dbf852422f1 Layer already exists 0.4s
```

B. Aprovisionamiento de la Web App en modo Multi-Container

```
# Crear el App Service Plan (si no lo tienes creado del ejercicio anterior)
az appservice plan create --name $APP_SERVICE_PLAN --resource-group
$RESOURCE_GROUP --sku B1 --is-linux

# Crear la Web App pasándole el docker-compose.yml directamente
az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name
```

```
$WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file  
docker-compose.yml
```

```
PS C:\Users\danie\azure-assignment> az appservice plan create --name $APP_SERVICE_PLAN --resource-group $RESOURCE_GROUP --sku B1 --is-linux
>>
{
  "asyncScalingEnabled": false,
  "elasticScaleEnabled": false,
  "extendedLocation": null,
  "freeOfferExpirationTime": null,
  "geoRegion": "Spain Central",
  "hostingEnvironmentProfile": null,
  "hyperV": false,
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.Web/serverfarms/plan-fastapi-gemini",
  "isSpot": false,
  "isXenon": false,
  "kind": "linux",
  "kubeEnvironmentProfile": null,
  "location": "spaincentral",
  "maximumElasticWorkerCount": 1,
  "maximumNumberOfWorkers": 3,
  "name": "plan-fastapi-gemini",
  "numberOfSites": 1,
  "numberOfWorkers": 1,
  "perSiteScaling": false,
  "provisioningState": "Succeeded",
  "reserved": true,
  "resourceGroup": "gruporecursosia",
  "sku": {
    "capabilities": null,
    "capacity": 1,
    "family": "B",
    "locations": null,
    "name": "B1",
    "size": "B1",
    "skuCapacity": null,
    "tier": "Basic"
  },
  "spotExpirationTime": null,
  "status": "Ready",
  "subscription": "884c87e5-7027-435d-905b-1722c9bcfddc",
  "tags": null,
  "targetWorkerCount": 0,
  "targetWorkerSizeId": 0,
  "type": "Microsoft.Web/serverfarms",
  "workerTierName": null,
  "zoneRedundant": false
}
PS C:\Users\danie\azure-assignment> ^C
```

```
PS C:\Users\danie\azure-assignment> az webapp create --resource-group $RESOURCE_GROUP --plan $APP_SERVICE_PLAN --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml
Webapp 'webapp-fastapi-gemini' already exists. The command will use the existing app's settings.
vnet_route_all_enabled is not a known attribute of class <class 'azure.mgmt.web.v2024_11_01.models._models_py3.Site'> and will be ignored
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "1fE448438D76E83CD089F2F8DC94FD32201CA23F2A99F0C844D6FA3298A1AE47",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "webapp-fastapi-gemini.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,
  "enabledHostNames": [
    "webapp-fastapi-gemini.azurewebsites.net",
    "webapp-fastapi-gemini.scm.azurewebsites.net"
  ],
  "endToEndEncryptionEnabled": false,
  "extendedLocation": null,
  "ftpPublishingUrl": "https://waws-prod-esc-003.ftp.azurewebsites.windows.net/site/wwwroot",
  "functionAppConfig": null,
  "hostNameSslStates": [
    {
      "certificateResourceId": null,
      "hostType": "Standard",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    },
    {
      "certificateResourceId": null,
      "hostType": "Repository",
      "ipBasedSslResult": null,
      "ipBasedSslState": "NotConfigured",
      "name": "webapp-fastapi-gemini.scm.azurewebsites.net",
      "sslState": "Disabled",
      "thumbprint": null,
      "toUpdate": null,
      "toUpdateIpBasedSsl": null,
      "virtualIPv6": null,
      "virtualIp": null
    }
  ]
}
```

C. Configuración de Variables en Azure

```
# Vincular las credenciales del ACR a la Web App (usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group
$RESOURCE_GROUP --container-registry-url "https://$ACR_NAME.azurecr.io" --
container-registry-user $ACR_NAME --container-registry-password $ACR_PASSWORD

# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-
Object { $_.ToString().Split("=",2)[1] })

# Definir la API Key de Gemini
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings GEMINI_API_KEY=$GEMINI_API_KEY

# Configurar el puerto público: Indicamos a Azure que envíe el tráfico al puerto
80 (Frontend)
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings WEBSITES_PORT=80

# Configurar comunicación interna: En Azure Multi-container los contenedores se
comunican por localhost
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings BACKEND_URL="http://localhost:8080"

# Iniciar la Web App (asegura el arranque si la aplicación está detenida)
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

```

PS C:\Users\danie\azure-assignment> az webapp config container set --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP --docker-custom-image-name $IMAGE_TAG --docker-registry-server-url "https://$ACR_NAME.azurecr.io" --docker-registry-server-user $ACR_NAME --docker-registry-server-password $ACR_PASSWORD
Option '--docker-custom-image-name' has been deprecated and will be removed in a future release. Use '--container-image-name' instead.
Option '--docker-registry-server-url' has been deprecated and will be removed in a future release. Use '--container-registry-url' instead.
Option '--docker-registry-server-user' has been deprecated and will be removed in a future release. Use '--container-registry-user' instead.
Option '--docker-registry-server-password' has been deprecated and will be removed in a future release. Use '--container-registry-password' instead.
[
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": "https://planservicioia.azurecr.io"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": "planservicioia"
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_CUSTOM_IMAGE_NAME",
    "value": "DOCKER|planservicioia.azurecr.io/fastapi-gemini:v1"
  }
]

PS C:\Users\danie\azure-assignment> az webapp config appsettings set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --settings WEBSITES_PORT=8080 GEMINI_API_KEY="AIzaSyC6nC-yreaSigMGrLnAwMj
d6Q@vwnoigh4"
[
  {
    "name": "DOCKER_REGISTRY_SERVER_URL",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_USERNAME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "DOCKER_REGISTRY_SERVER_PASSWORD",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITES_PORT",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "GEMINI_API_KEY",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "BACKEND_URL",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITES_ENABLE_APP_SERVICE_STORAGE",
    "slotSetting": false,
    "value": null
  }
]

```

D. Prueba del Despliegue en Azure Una vez que App Service haya descargado las imágenes y levantado los contenedores (esto puede tomar 2-3 minutos la primera vez), puedes probar que el flujo multi-contenedor funciona haciendo una llamada a tu dominio .azurewebsites.net:

```

# Realizar petición de prueba a la API Frontend desplegada
Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?
prompt=Test_Compose_Azure" -UseBasicParsing

```

```

PS C:\Users\danie\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Test_Compose_Azure"
>>

Advertencia de seguridad: riesgo de ejecución de script
Invoke-WebRequest analiza el contenido de la página web. El código de script de la página web se puede ejecutar cuando se analiza la página.
  ACCIÓN RECOMENDADA:
  Usa el modificador -UseBasicParsing para evitar la ejecución de código de script.

¿Quieres continuar?

[S] Sí [O] Sí a todo [N] No [Y] No a todo [U] Suspender [?] Ayuda (el valor predeterminado es "N"): s

StatusCode      : 200
StatusDescription : OK
Content          : {"prompt":"Test_Compose_Azure","text":"The phrase \"Test_Compose_Azure\" isn't a standard Azure service name or a specific command. Instead, it sounds like a task description
                  or a scenario. It most li...
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 8379
                  Content-Type: application/json
                  Date: Wed, 03 Jun 2026 16:41:21 GMT
                  Set-Cookie: ARRAffinity=279e2686d6d985a92f0e5544b67f0addb363f77898a222be35d50034f8df4ba;Pat...
Forms            : {}
Headers          : {[Content-Length, 8379], [Content-Type, application/json], [Date, Wed, 03 Jun 2026 16:41:21 GMT], [Set-Cookie, ARRAffinity=279e2686d6d985a92f0e5544b67f0addb363f77898a222be35d5
                  0034f8df4ba;Path=/;HttpOnly;Secure;Domain=webapp-fastapi-gemini.azurewebsites.net,ARRAffinitySameSite=279e2686d6d985a92f0e5544b67f0addb363f77898a222be35d50034f8df4ba;Path=/;
                  HttpOnly;SameSite=None;Secure;Domain=webapp-fastapi-gemini.azurewebsites.net]...}
Images           : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 8379

```

(Nota: Si obtienes un error inicial o un tiempo de espera agotado, espera un par de minutos a que los contenedores terminen de arrancar o ejecuta `az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP` para forzar un reinicio).

(Nota para actualizaciones: Si realizas modificaciones en el archivo `docker-compose.yml` localmente y deseas subirlas a Azure, debes volver a cargar la configuración en la Web App ejecutando: `az webapp config container set --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file docker-compose.yml`).

2.4. Solución de Problemas y Cambios de Código (Troubleshooting)

Durante la migración a la arquitectura multi-contenedor, nos encontramos con varios errores que requirieron los siguientes ajustes en el código y en Azure:

1. Tolerancia a fallos en el Backend (`backend/app.py`)

Problema: Si la variable `GEMINI_API_KEY` no se pasaba correctamente (o tardaba en inyectarse), la llamada a `genai.Client()` fallaba y el contenedor `backend` fallaba de forma abrupta en el arranque, provocando que el Frontend no pudiera conectarse. **Solución:** Se envolvió la inicialización en un bloque `try/except` para que el contenedor siempre arranque y devuelva el error de forma controlada al usuario.

```

try:
    client = genai.Client()
except Exception as e:
    client = None
    print(f"Warning: genai.Client initialization failed: {e}")

```

2. Timeout y Traza de Errores en Frontend (`frontend/app.py`)

Problema: Las peticiones desde el frontend al backend sufrían timeouts silenciosos si Gemini tardaba en responder, devolviendo un error `{"detail": ""}` incomprensible. **Solución:** Se añadió un `timeout=60.0`

explícito en `httpx.AsyncClient(timeout=60.0)` y se cambió la captura de excepciones para usar `repr(e)` y devolver detalles claros:

```
except Exception as e:
    raise HTTPException(status_code=500, detail=repr(e))
```

3. Soporte de variables en Docker Compose (Azure App Service)

Problema: Azure App Service Multi-container no soporta el formato `${ACR_NAME}` en la propiedad `image:` del archivo de compose (provocaba el error `Application Error`). **Solución:** Se hardcodearon directamente los nombres de imagen en el `docker-compose.yml` con el valor del ACR (`planservicioia.azurecr.io/...`), eliminando la dependencia de variables dinámicas en tiempo de despliegue.

4. Enrutamiento del puerto público en Azure

Problema: En el despliegue de un solo contenedor, Azure enrutaba el tráfico al puerto 8080 (`WEBSITES_PORT=8080`). Al cambiar a multi-contenedor, nuestro nuevo Frontend público escucha por el puerto 80, provocando un fallo en el balanceador. **Solución:** Se forzó el reseteo de la variable `WEBSITES_PORT` para redirigir el tráfico externo correctamente al puerto 80.

```
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings WEBSITES_PORT=80
```

5. Comunicación de red interna entre contenedores en Azure

Problema: Localmente los contenedores se comunicaban vía `http://backend:8080`, pero en Azure fallaba con `ConnectError`. Esto ocurre porque Azure Web App For Containers (Compose) hace que todos los contenedores compartan el mismo espacio de red (namespaces). **Solución:** Se modificó la URL del backend en Azure para que el Frontend llamara a `localhost` en lugar del nombre del servicio.

```
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --settings BACKEND_URL="http://localhost:8080"
```

6. Extracción y análisis de logs en Azure

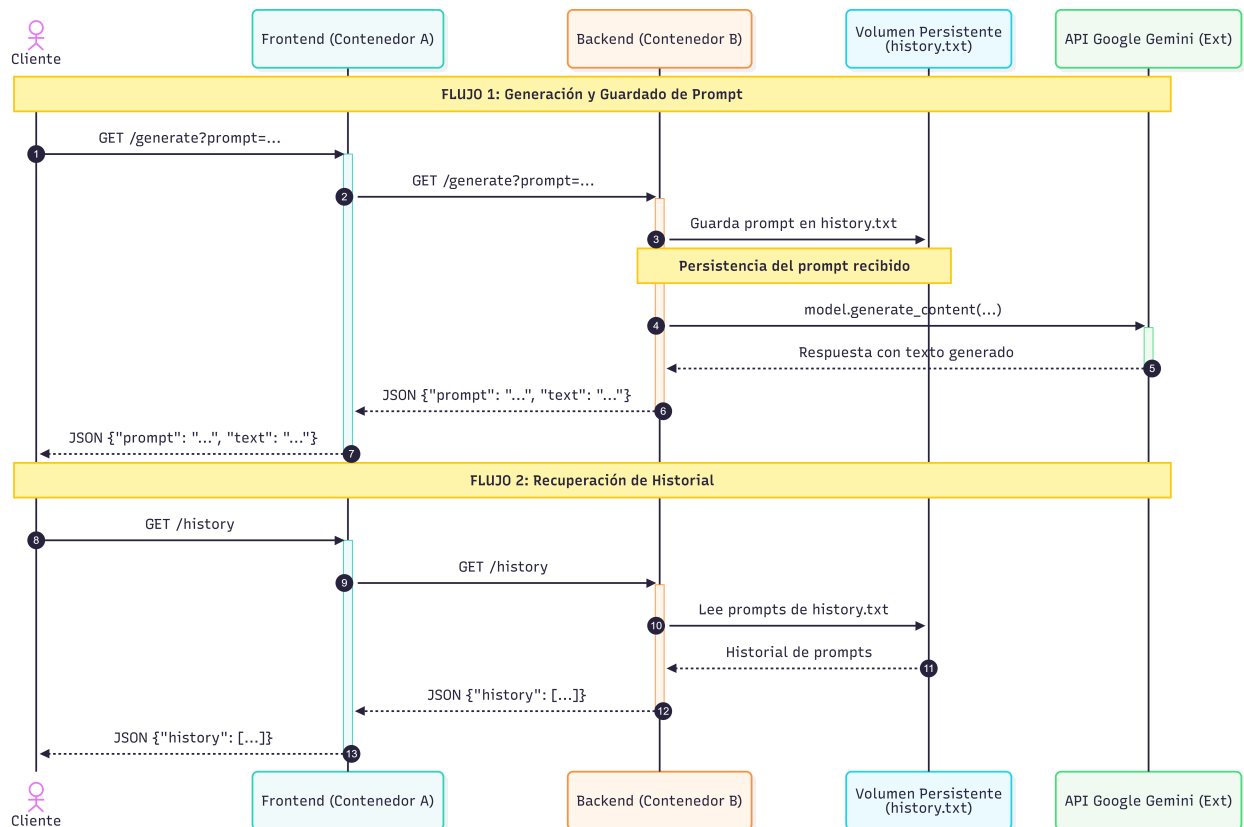
Problema: Durante los errores `( Application Error`, la consola local no provee información de por qué fallaron los contenedores en el despliegue. **Solución:** Usamos comandos de la CLI de Azure para diagnosticar qué ocurría internamente. Para descargar todos los logs como un archivo ZIP local usamos:

```
az webapp log download --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

Adicionalmente, para ver los logs en tiempo real por la consola (muy útil durante arranques lentos o reinicios):

```
az webapp log tail --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

Bloque 3: Despliegue Compose con Persistencia



Para asegurar que las consultas realizadas a la aplicación no se pierdan cuando se reinician los contenedores o se vuelve a desplegar el servicio en la nube, se ha implementado un mecanismo de almacenamiento persistente montando un recurso compartido de archivos de Azure (Azure File Share) directamente en los contenedores.

El servicio Backend guarda cada prompt en formato de texto plano dentro de la ruta `/app/data/history.txt`. Esta ruta está mapeada a un almacenamiento externo y permanente.

3.1. Aprovisionamiento de Azure Files y Conexión con la Web App

Sigue estos pasos en la terminal PowerShell para crear el almacenamiento persistente y conectarlo a tu Web App:

Paso 1: Definir variables de entorno adicionales

```
# Variables del entorno de Azure (redefinir si es una sesión de terminal nueva)
$RESOURCE_GROUP = "gruporecursosia"
$LOCATION         = "spaincentral"
$WEB_APP_NAME    = "webapp-fastapi-gemini"
```



```
# Variables para el almacenamiento persistente
$STORAGE_ACCOUNT = "almacenamientoappia" # Debe ser un nombre único globalmente
$SHARE_NAME      = "archivos-texto"
$MOUNT_PATH      = "/app/data" # Ruta dentro del contenedor donde se guardará el
historial
```

Paso 2: Crear la Infraestructura de Almacenamiento

```
# 1. Crear la Cuenta de Almacenamiento (Storage Account)
az storage account create --name $STORAGE_ACCOUNT --resource-group $RESOURCE_GROUP
--location $LOCATION --sku Standard_LRS

# 2. Crear el recurso compartido de archivos (File Share)
az storage share-rm create --resource-group $RESOURCE_GROUP --storage-account
$STORAGE_ACCOUNT --name $SHARE_NAME --quota 5
```

```

PS C:\Users\danie\azure-assignment> $STORAGE_ACCOUNT = "almacenamientoappia" # Debe ser un nombre único globalmente
> $SHARE_NAME = "archivos-texto"
> $MOUNT_PATH = "/app/data" # Ruta dentro del contenedor donde se guardará el historial
PS C:\Users\danie\azure-assignment> az storage account create --name $STORAGE_ACCOUNT --resource-group $RESOURCE_GROUP --location $LOCATION --sku Standard_LRS
A storage account with the provided name almacenamientoappia is found. Will continue to update the existing account.
{
  "accessTier": "Hot",
  "accountMigrationInProgress": null,
  "allowBlobPublicAccess": false,
  "allowCrossTenantReplication": false,
  "allowSharedKeyAccess": null,
  "allowedCopyScope": null,
  "azureFilesIdentityBasedAuthentication": null,
  "blobRestoreStatus": null,
  "creationTime": "2026-06-03T18:39:53.057487+00:00",
  "customDomain": null,
  "defaultToOAuthAuthentication": null,
  "dnsEndpointType": null,
  "dualStackEndpointPreference": null,
  "enableExtendedGroups": null,
  "enableHttpsTrafficOnly": true,
  "enableWfsV3": null,
  "encryption": {
    "encryptionIdentity": null,
    "keySource": "Microsoft.Storage",
    "keyVaultProperties": null,
    "requireInfrastructureEncryption": null,
  },
  "services": {
    "blob": {
      "enabled": true,
      "keyType": "Account",
      "lastEnabledTime": "2026-06-03T18:39:53.139327+00:00"
    },
    "file": {
      "enabled": true,
      "keyType": "Account",
      "lastEnabledTime": "2026-06-03T18:39:53.139327+00:00"
    },
    "queue": null,
    "table": null
  },
  "extendedLocation": null,
  "failoverInProgress": null,
  "geoPriorityReplicationStatus": null,
  "geoReplicationStats": null,
  "id": "/subscriptions/884c87e5-7027-435d-905b-1722c9bcfddc/resourceGroups/gruporecursosia/providers/Microsoft.Storage/storageAccounts/almacenamientoappia",
  "identity": null,
  "immutableStorageWithVersioning": null,
  "isHnsEnabled": null,
  "isLocalUserEnabled": null,
  "isSftpEnabled": null,
  "isSkuConversionBlocked": null,
  "keyCreationTime": {
    "key1": "2026-06-03T18:39:53.129152+00:00",
    "key2": "2026-06-03T18:39:53.129152+00:00"
  },
  "keyPolicy": null,
  "kind": "StorageV2",
  "largeFileSharesState": null,
  "lastGeoFailoverTime": null,
  "location": "spaincentral",
  "minimumTlsVersion": "TLS1_0",
  "name": "almacenamientoappia",
  "networkRuleSet": {
    "bypass": "AzureServices",
    "defaultAction": "Allow",
    "ipRules": [],
  },
}

```

Paso 3: Conectar el Almacenamiento a la Web App

1. Obtener la clave de acceso del Storage Account

```
$STORAGE_KEY = (az storage account keys list --resource-group $RESOURCE_GROUP --
account-name $STORAGE_ACCOUNT --query "[0].value" --output tsv)
```

2. (Opcional) Eliminar un montaje previo con el mismo ID si existe

```
az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente
```

3. (Opcional) Limpiar el historial previo en la nube para empezar con una demo limpia (si no existe el archivo, dará error, el cual se puede ignorar)

```
az storage file delete --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --
account-key $STORAGE_KEY --path history.txt
```

4. Montar el almacenamiento persistente en la Web App

```
az webapp config storage-account add --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --
account-name $STORAGE_ACCOUNT --share-name $SHARE_NAME --access-key $STORAGE_KEY -
-mount-path $MOUNT_PATH
```

4. Reiniciar la Web App para que el montaje surta efecto

```
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

```
PS C:\Users\danie\azure-assignment> az storage share create --resource-group $RESOURCE_GROUP --storage-account $STORAGE_ACCOUNT --name $SHARE_NAME --quota 5
{
  "id": "/subscriptions/884c87e5-7827-435d-985b-1722c9bcfdcd/resourceGroups/grupo recursosia/providers/Microsoft.Storage/storageAccounts/almacenamientoappia/fileServices/default/shares/archivos-texto",
  "name": "archivos-texto",
  "resourceGroup": "grupo recursosia",
  "shareQuota": 5,
  "type": "Microsoft.Storage/storageAccounts/fileServices/shares"
}
PS C:\Users\danie\azure-assignment> az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --custom-id AlmacenamientoPersistente
Storage account access keys have been redacted. Use 'az webapp config storage-account list' to view.
{}
PS C:\Users\danie\azure-assignment> az webapp config storage-account add --resource-group $RESOURCE_GROUP --name $WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --account-name $STORAGE_ACCOUNT --share-na
me $SHARE_NAME --access-key $STORAGE_KEY --mount-path $MOUNT_PATH
Storage account access keys have been redacted. Use 'az webapp config storage-account list' to view.
{
  "AlmacenamientoPersistente": {
    "accessKey": null,
    "accountName": "almacenamientoappia",
    "mountPath": "/app/data",
    "protocol": "Smb",
    "shareName": "archivos-texto",
    "state": "Ok",
    "type": "AzureFiles"
  }
}
PS C:\Users\danie\azure-assignment>
```

3.2. Ajuste al Docker Compose (`docker-compose.yml`)

Para soportar las pruebas y persistencia en local, se actualiza el `docker-compose.yml` añadiendo el volumen persistente al servicio `backend` y etiquetando las imágenes como `v2`. El archivo completo queda así:

```
version: '3.8'

services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v2
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"
    volumes:
      - AlmacenamientoPersistente:/app/data

  frontend:
    build: ./frontend
    image: planservicioia.azurecr.io/fastapi-frontend:v2
    environment:
      - BACKEND_URL=http://backend:8080
    ports:
      - "80:80"
    depends_on:
```

```
- backend
```

```
volumes:  
  AlmacenamientoPersistente:
```

- **En Local:** docker-compose crea y gestiona automáticamente un volumen local llamado `AlmacenamientoPersistente` que se monta en `/app/data` y persiste los datos de tus consultas.
- **En Azure:** Azure App Service intercepta el nombre del volumen (`AlmacenamientoPersistente`) y lo asocia automáticamente con el montaje de Azure Files que tiene el mismo `custom-id` (Paso 3), guardando el archivo de texto directamente en la cuenta de almacenamiento en la nube sin depender del almacenamiento efímero del contenedor.

3.3. Pruebas de Persistencia en Local

⚠ **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → *"Docker Desktop is running"*).

1. Levanta los contenedores en local:

```
docker-compose up --build -d
```

2. Envía prompts de prueba al endpoint público de generación (puerto 80):

```
Invoke-WebRequest "http://localhost:80/generate?prompt=Local_Prompt_1" -  
UseBasicParsing  
Invoke-WebRequest "http://localhost:80/generate?prompt=Local_Prompt_2" -  
UseBasicParsing
```

3. Verifica que se ha creado el historial accediendo al endpoint `/history`:

```
(Invoke-WebRequest "http://localhost:80/history" -UseBasicParsing).Content  
# Salida esperada: {"history":["Local_Prompt_1"]}
```

```

PS C:\Users\daniel\azure-assignment> docker-compose up --build -d
time="2026-06-03T23:20:44Z" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 2.1s (20/20) #STDERR
=> [internal] load local bake definitions 0.0s
=> reading from stdin 1.000s 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> transferring Dockerfile: 500s 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> transforming Dockerfile: 407s 0.0s
=> [backend internal] load metadata for docker.io/library/python:3.11-slim 1.1s
=> [frontend internal] load .dockerignore 0.0s
=> transforming context: 2s 0.0s
=> [backend internal] load .dockerignore 0.0s
=> transforming context: 2s 0.0s
=> [backend 1/5] FROM docker.io/library/python:3.11-slim@sha256:abab8960c4e1546a813c2081c1848a807 0.0s
=> resolve docker.io/library/python:3.11-slim@sha256:abab8960c4e1546a813c2081c1848a807 0.0s
=> [frontend internal] load build context 0.0s
=> transforming context: 63s 0.0s
=> [backend internal] load build context 0.0s
=> transforming context: 63s 0.0s
=> CACHED [frontend 2/5] WORKDIR /app 0.0s
=> CACHED [backend 3/5] COPY requirements.txt . 0.0s
=> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [backend 5/5] COPY app.py . 0.0s
=> CACHED [frontend 3/5] COPY requirements.txt . 0.0s
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [frontend 5/5] COPY app.py . 0.0s
=> [frontend] exporting to image 0.2s
=> exporting layers 0.0s
=> exporting manifest sha256:85bc3108172161f0d0f0ba308f11bdf1d3e312cc0d81493cd7784bfff 0.0s
=> exporting config sha256:b6f12ab30400976d7142bdc8a72076f1c62f7067274f92f21a182433c 0.1s
=> exporting attestation manifest sha256:42e148d834b6151d9b8c14c7e70c1919c3b0b08a3c7ad1dcf5d 0.1s
=> exporting manifest list sha256:1560805963f57a14fc0c583bccc88040994221c15109f6c0a8888000196 0.0s
=> naming to plaveservicia.azurecr.io/fastapi-backend-v2 0.0s
=> unpacking to plaveservicia.azurecr.io/fastapi-backend-v2 0.0s
=> [backend] resolving provenance for metadata file 0.0s
=> [backend] resolving provenance for metadata file 0.0s
[+] Running 5/6
✔ plaveservicia.azurecr.io/fastapi-backend-v2 Built 0.0s
✔ plaveservicia.azurecr.io/fastapi-frontend-v2 Built 0.0s
✔ Network azure-assignment_default Created 0.1s
✔ Volume azure-assignment_alacamentopersistente Created 0.0s
✔ Container azure-assignment-backend-1 Started 0.6s
✔ Container azure-assignment-frontend-1 Started 0.9s
PS C:\Users\daniel\azure-assignment> Invoke-WebRequest "http://localhost:80/generateprompt?Local_Prompt_1"

Statuscode : 200
StatusDescription : OK
Content : {"prompt":"Local_Prompt_1","text":"\Local_Prompt_1\" seems to be a placeholder or a general reference. \nInfo help you with it, I need more information. Could you please tell me \nWhat is the ..."}
RawContent : HTTP/1.1 200 OK
Content-Length : 655
Content-Type : application/json
Date: Wed, 03 Jun 2026 21:20:17 GMT
Server: uvicorn

{"prompt":"Local_Prompt_1","text":"\Local_Prompt_1\" seems to be a plac...
Forms : {}
Headers : [{"Content-Length": 655}, {"Content-Type": "application/json"}, {"Date": "Wed, 03 Jun 2026 21:20:17 GMT"}, {"Server": "uvicorn"}]
Images : {}
InputFields : {}
Links : {}

PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "http://localhost:80/history").Content ("history":["Local_Prompt_1"])
time="2026-06-03T23:20:53Z" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
✔ Container azure-assignment-frontend-1 Removed 0.5s
✔ Container azure-assignment-backend-1 Removed 0.7s
✔ Network azure-assignment_default Removed 0.4s
PS C:\Users\daniel\azure-assignment> docker-compose up -d
time="2026-06-03T23:21:01Z" level=warning msg="C:\Users\daniel\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
✔ Network azure-assignment_default Created 0.1s
✔ Container azure-assignment-backend-1 Started 0.6s
✔ Container azure-assignment-frontend-1 Started 0.8s
PS C:\Users\daniel\azure-assignment> (Invoke-WebRequest "http://localhost:80/history").Content ("history":["Local_Prompt_1"])
PS C:\Users\daniel\azure-assignment>

```

4. Detén y elimina por completo los contenedores:

```
docker-compose down
```

5. Vuelve a levantar el servicio:

```
docker-compose up -d
```

6. Consulta nuevamente el historial.

```
(Invoke-WebRequest "http://localhost:80/history" -UseBasicParsing).Content
# Salida esperada: {"history":["Local_Prompt_1"]}
```

Los datos deben seguir presentes puesto que están guardados en el archivo `./data/history.txt` de tu equipo local y sobreviven a la recreación de los contenedores.

3.4. Pruebas de Persistencia en Azure

1. Reconstruye y sube las imágenes al ACR con los nuevos cambios de código:

```
$env:ACR_NAME=$ACR_NAME
docker-compose build
docker-compose push
```

2. Actualiza la configuración de la Web App con el nuevo `docker-compose.yml` (que incluye el volumen) y fuerza el reinicio:

```
# Subir la nueva configuración del compose a Azure (incluye el volumen en
backend)
az webapp config container set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-
file docker-compose.yml

# Reiniciar para descargar las nuevas imágenes y aplicar el montaje
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

3. Envía una consulta a la Web App en la nube:

```
Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/generate?
prompt=Azure_Prompt_1" -UseBasicParsing
```

4. Consulta el endpoint de historial para comprobar que ha registrado la consulta:

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history"
-UseBasicParsing).Content
# Salida esperada: {"history":["Azure_Prompt_1"]}
```

5. (Opcional) Verifica la persistencia listando los archivos en Azure File Share directamente mediante Azure CLI:

```
# Listar archivos en el File Share
az storage file list --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT
--account-key $STORAGE_KEY --output table

# Descargar y mostrar el contenido de history.txt
az storage file download --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --path history.txt --dest .
Get-Content history.txt
Remove-Item history.txt
```

6. Forza un reinicio de la Web App en Azure para simular un fallo o actualización del servicio:

```
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

7. Vuelve a realizar la consulta al historial:

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/history"
-UseBasicParsing).Content
```

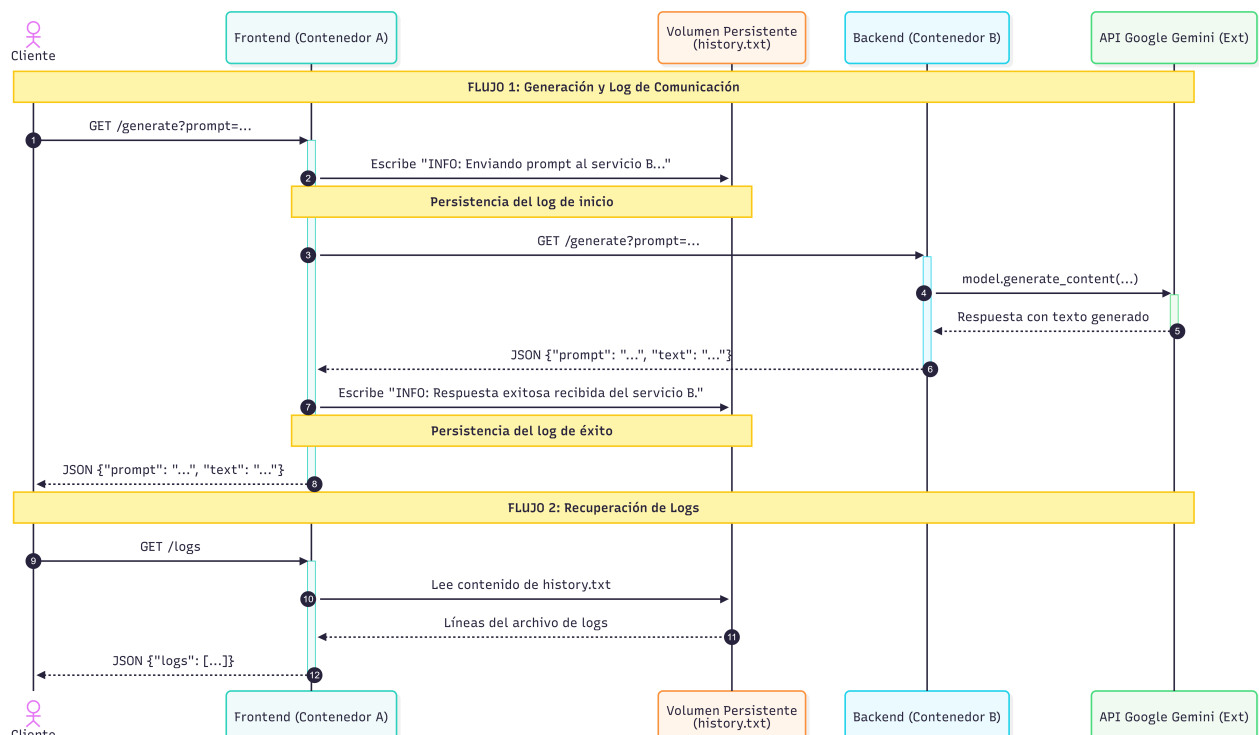
Resultado: El endpoint debe seguir devolviendo `{"history":["Azure_Prompt_1"]}` demostrando que el archivo se almacena en el volumen persistente de Azure Files y no se pierde tras reiniciar el servicio.

```
PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/history").Content
{"history":["Azure_Prompt_1","Azure_Prompt_2"]}
PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/history").Content
{"history":["Azure_Prompt_1","Azure_Prompt_2"]}
PS C:\Users\danie\azure-assignment> az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/history").Content
{"history":["Azure_Prompt_1","Azure_Prompt_2"]}
```

(Nota: Es posible que aparezcan más entradas en la lista del historial si ya habías lanzado peticiones de prueba anteriormente).

Bloque 4: Despliegue Final con Persistencia y Prueba de Interconexión

4.1. Diagrama de Arquitectura Final



En esta nueva iteración de la arquitectura (imágenes etiquetadas con la versión **v3**), se ha modificado la demostración de la comunicación de red entre contenedores de la siguiente forma:

1. **Reemplazo de `/history`:** Se eliminó la anterior funcionalidad de historial de prompts que gestionaba el Backend en `history.txt`.
2. **Generación de Logs en Frontend:** Ahora, el Frontend (Contenedor A) usa la librería estándar `logging` de Python para registrar cada comunicación con el Backend (Contenedor B) en el archivo `history.txt`.
3. **Endpoint `/logs`:** Se habilitó un nuevo endpoint en el Frontend (`GET /logs`) para leer de forma local el archivo de logs `history.txt` y retornar los registros del flujo de peticiones.
4. **Reubicación de Volumen:** El volumen persistente `AlmacenamientoPersistente` ha sido reubicado en el Frontend para conservar el historial de logs ante reinicios.

4.2. Configuración en docker-compose (Versión v3)

El archivo `docker-compose.yml` actualiza las imágenes a la versión **v3** y traslada el volumen persistente al servicio `frontend`:

```
version: '3.8'

services:
  backend:
    build: ./backend
    image: planservicioia.azurecr.io/fastapi-backend:v3
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    expose:
      - "8080"

  frontend:
    build: ./frontend
    image: planservicioia.azurecr.io/fastapi-frontend:v3
    environment:
      - BACKEND_URL=http://backend:8080
    ports:
      - "80:80"
    depends_on:
      - backend
    volumes:
      - AlmacenamientoPersistente:/app/data

volumes:
  AlmacenamientoPersistente:
```

4.3. Definición de los Contenedores (Dockerfiles)

Para la construcción de los contenedores de Frontend y Backend de esta arquitectura final, se utilizan los siguientes archivos `Dockerfile`:

Backend (`backend/Dockerfile`)


```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 8080

# Run the app with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Frontend (frontend/Dockerfile)

```
# Use official Python slim image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements.txt first to leverage Docker cache
COPY requirements.txt .

# Install all dependencies from requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI app
COPY app.py .

# Expose FastAPI port
EXPOSE 80

# Run the app with Uvicorn on port 80
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "80"]
```

4.4. Pruebas de Logs y Persistencia en Local

⚠ **Prerequisito:** Comprueba que **Docker Desktop** está en ejecución antes de continuar (icono en la barra de tareas → "Docker Desktop is running").

1. Reconstruye y levanta los contenedores con las imágenes v3:

```
# Leer la API Key desde el archivo .env
$env:GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" |
ForEach-Object { $_.ToString().Split("=",2)[1] })
docker-compose up --build -d
```

2. Envía una consulta de prueba al frontend (puerto 80):

```
Invoke-WebRequest "http://localhost:80/generate?
prompt=Test_Communication_Logs"
```

3. Verifica la traza de logs de comunicación a través del endpoint /logs:

```
(Invoke-WebRequest "http://localhost:80/logs").Content
```

Salida esperada (en formato JSON):

```
{
  "logs": [
    "2026-06-04 20:50:44,123 - INFO - Enviando prompt al servicio B en la URL
http://backend:8080/generate",
    "2026-06-04 20:50:44,456 - INFO - Respuesta exitosa recibida del servicio
B."
  ]
}
```

4. Detén y elimina por completo los contenedores para validar la persistencia:

5. Vuelve a iniciar los contenedores (esta vez sin reconstruir la imagen):

6. Consulta nuevamente el endpoint de logs:

Resultado: El endpoint debe seguir devolviendo los registros de logs previos ("INFO - Enviando prompt..." y "INFO - Respuesta exitosa..."), demostrando que el archivo se almacena en el volumen persistente `AlmacenamientoPersistente` en tu equipo local y sobrevive a la recreación de los contenedores.

35 / 43

4.5. Instrucciones de Despliegue en Azure (v3)

Para desplegar esta nueva configuración (imágenes **v3** y persistencia en el Frontend) en Azure, ejecuta los siguientes comandos en tu terminal de PowerShell:

A. Construcción y subida de las nuevas imágenes v3

```
# Exportamos el nombre del ACR para etiquetar las imágenes v3
$env:ACR_NAME=$ACR_NAME

# Autenticarse en el ACR antes de hacer push (usando password-stdin por seguridad)
echo $ACR_PASSWORD | docker login "$ACR_NAME.azurecr.io" --username $ACR_NAME --password-stdin

# Construimos y subimos las imágenes
docker-compose build
docker-compose push
```

```

PS C:\Users\danie\azure-assignment> $env:ACR_NAME=$ACR_NAME
>>
PS C:\Users\danie\azure-assignment> docker-compose build
>>
time="2026-06-04T22:41:40+02:00" level=warning msg="C:\Users\danie\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Building 2.3s (20/20) FINISHED
-> [internal] load local bake definitions                                0.0s
-> -> reading from stdin 1.08s                                         0.0s
-> [backend internal] load build definition from Dockerfile             0.0s
-> -> transferring dockerfile: 508B                                     0.0s
-> [frontend internal] load build definition from Dockerfile           0.0s
-> -> transferring dockerfile: 487B                                      0.0s
-> [backend internal] load metadata for docker.io/library/python:3.11-slim 1.0s
-> [backend internal] load .dockerignore                                0.0s
-> -> transferring context: 2B                                           0.0s
-> [frontend internal] load .dockerignore                               0.0s
-> -> transferring context: 2B                                           0.0s
-> [frontend 1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833e22093cb840908979487a9fc8e6e38295747e48ac9 0.0s
-> -> resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a833e22093cb840908979487a9fc8e6e38295747e48ac9 0.0s
-> [backend internal] load build context                                0.0s
-> -> transferring context: 63B                                          0.0s
-> [frontend internal] load build context                               0.0s
-> -> transferring context: 63B                                          0.0s
-> CACHED [frontend 2/5] WORKDIR /app                                   0.0s
-> CACHED [backend 3/5] COPY requirements.txt .                          0.0s
-> CACHED [backend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [backend 5/5] COPY app.py .                                    0.0s
-> CACHED [frontend 3/5] COPY requirements.txt .                        0.0s
-> CACHED [frontend 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [frontend 5/5] COPY app.py .                                  0.0s
-> [frontend] exporting to image                                        0.2s
-> -> exporting layers                                                  0.0s
-> -> exporting manifest sha256:9d4abc5d8f3f2c942f4b62b469caf44c9be1824901bcb03f77a18fe583276bac 0.0s
-> -> exporting config sha256:9b18d7854f8623b44bfdfafae9e10043b2b655834be4a82c342a4a8c7e19cf4 0.0s
-> -> exporting attestation manifest sha256:b39c58f80c28efc34aafdb9752772a8f192664807f2da7c5124657cc282fcl9 0.1s
-> -> exporting manifest list sha256:1ffb8bf6f1486c443d82b70f018984156bf3377ab9087f34f1a85e88e0db4 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-frontend:v3 0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-frontend:v3 0.0s
-> [backend] exporting to image                                        0.2s
-> -> exporting layers                                                  0.0s
-> -> exporting manifest sha256:0e496735b634074092251ed3be1f768ba7ac2852d27e6785c6395333be1f4ded 0.0s
-> -> exporting config sha256:fal6f9fcec97c82ecf58c2c1b41f2d6e698adaf4de1af6f88775495cecc8cd53 0.0s
-> -> exporting attestation manifest sha256:cb87b7dd204c8317f2a093f00667089ff8213978c2240870c83a1f0eb8824b5e 0.1s
-> -> exporting manifest list sha256:3fa121a3f5b14aa6dffa18a6a85c29d0e35cd5d1695a52c39ac1cc7c0513963 0.0s
-> -> naming to planservicioia.azurecr.io/fastapi-backend:v3 0.0s
-> -> unpacking to planservicioia.azurecr.io/fastapi-backend:v3 0.0s
-> [frontend] resolving provenance for metadata file                   0.0s
-> [backend] resolving provenance for metadata file                   0.0s
[*] Building 2/2
✓ planservicioia.azurecr.io/fastapi-frontend:v3 Built 0.0s
✓ planservicioia.azurecr.io/fastapi-backend:v3 Built 0.0s
PS C:\Users\danie\azure-assignment> docker-compose push
>>
time="2026-06-04T22:41:40+02:00" level=warning msg="C:\Users\danie\azure-assignment\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Pushing 18/18
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: b8278f1c19f3 Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 571cd5d75cda Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 797d495f2c68 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 5b4deff92fc4 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: b727f22bc825 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 45086ceccc99 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 8649771fee17 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 5dbf852422f1 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-frontend:v3: 3e75f8258abc Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5542906c1817 Pushed 0.5s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 97ffe04ed1b0 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5b4deff92fc4 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 797d495f2c68 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 5dbf852422f1 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 8649771fee17 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: 45086ceccc99 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: e285e1798bd9 Layer already exists 0.3s
✓ Pushing planservicioia.azurecr.io/fastapi-backend:v3: bf45fbbe4797 Pushed 0.5s
PS C:\Users\danie\azure-assignment>

```

B. Actualización de la Web App en Azure

```

# Definir variables de entorno (por si inicias una nueva sesión de terminal)
$RESOURCE_GROUP = "gruporecursosia"
$ACR_NAME        = "planservicioia"
$WEB_APP_NAME    = "webapp-fastapi-gemini"
$STORAGE_ACCOUNT = "almacenamientoappia"
$SHARE_NAME      = "archivos-texto"
$ACR_PASSWORD    = (az acr credential show --name $ACR_NAME --query
"passwords[0].value" -o tsv)
$STORAGE_KEY     = (az storage account keys list --resource-group $RESOURCE_GROUP
--account-name $STORAGE_ACCOUNT --query "[0].value" --output tsv)

# Forzar a Azure a leer la nueva configuración del docker-compose.yml con las
imágenes v3
az webapp config container set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --multicontainer-config-type compose --multicontainer-config-file

```

```

docker-compose.yml

# Reconectar las credenciales del ACR a la Web App (necesario tras actualizar el
compose, usando parámetros actualizados)
az webapp config container set --name $WEB_APP_NAME --resource-group
$RESOURCE_GROUP `
  --container-registry-url "https://$ACR_NAME.azurecr.io" `
  --container-registry-user $ACR_NAME `
  --container-registry-password $ACR_PASSWORD

# Leer la API Key desde el archivo .env
$GEMINI_API_KEY = (Get-Content .env | Select-String "GEMINI_API_KEY" | ForEach-
Object { $_.ToString().Split("=",2)[1] })

# Configurar variables de entorno: API Key, puerto público y URL interna del
backend
az webapp config appsettings set --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME `
  --settings GEMINI_API_KEY=$GEMINI_API_KEY WEBSITES_PORT=80
BACKEND_URL="http://localhost:8080"

```

Nota: Azure App Service Multi-container hace que todos los contenedores compartan el mismo espacio de red, por lo que el Frontend debe llamar al Backend mediante `http://localhost:8080` en lugar de `http://backend:8080` (nombre de servicio de Compose). Esto queda reflejado en la variable `BACKEND_URL`.

C. Configuración del Almacenamiento Persistente en Azure Dado que el volumen

`AlmacenamientoPersistente` ahora se monta sobre el Frontend (para guardar `history.txt`), Azure se encargará de mapear automáticamente el File Share al contenedor Frontend gracias al ID coincidente. Para asociarlo (eliminando previamente cualquier montaje con el mismo ID si existe):

```

# 1. Eliminar un montaje previo con el mismo ID si existe
az webapp config storage-account delete --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente

# 2. (Opcional) Limpiar el historial previo en la nube para empezar con una demo
limpia (si no existe el archivo, dará error, el cual se puede ignorar)
az storage file delete --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT --
account-key $STORAGE_KEY --path history.txt

# 3. Asociar la cuenta de almacenamiento a la Web App
az webapp config storage-account add --resource-group $RESOURCE_GROUP --name
$WEB_APP_NAME --custom-id AlmacenamientoPersistente --storage-type AzureFiles --
account-name $STORAGE_ACCOUNT --share-name $SHARE_NAME --access-key $STORAGE_KEY -
-mount-path /app/data

```

D. Reinicio y Verificación

1. Realiza el reinicio inicial para aplicar la configuración de almacenamiento y descargar las nuevas imágenes (si la Web App está detenida, utiliza `start` en lugar de `restart`):

2. Envía una petición de generación de contenido (espera un par de minutos a que los contenedores hayan arrancado):

(Nota: Durante el periodo de arranque, es completamente normal y esperable que la petición falle inicialmente con un error de conexión `{"detail": "ConnectError('ALL connection attempts failed')"}` mientras el contenedor backend termina de iniciar. Espera uno o dos minutos y vuelve a realizar la llamada).

39 / 43

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/logs" -
UseBasicParsing).Content
```

4. (Opcional) Verifica que el archivo de logs se ha creado físicamente en el almacenamiento de Azure Files mediante CLI:

```
# Listar archivos en el File Share
az storage file list --share-name $SHARE_NAME --account-name $STORAGE_ACCOUNT
--account-key $STORAGE_KEY --output table

# Descargar y ver el contenido de history.txt
az storage file download --share-name $SHARE_NAME --account-name
$STORAGE_ACCOUNT --account-key $STORAGE_KEY --path history.txt --dest .
Get-Content history.txt
Remove-Item history.txt
```

5. Fuerza un reinicio de la Web App en Azure para simular la destrucción y recreación de los contenedores:

```
az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
```

6. Realiza de nuevo la consulta al endpoint de logs (espera el arranque):

```
(Invoke-WebRequest "https://webapp-fastapi-gemini.azurewebsites.net/logs" -
UseBasicParsing).Content
```

Resultado esperado: El endpoint debe seguir devolviendo los registros de logs previos ("INFO - Enviando prompt..." e "INFO - Respuesta exitosa..."), demostrando que el historial persistió gracias al montaje de Azure Files.


```

PS C:\Users\danie\azure-assignment> az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
>>
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
>>
Invoke-WebRequest:
{
  "detail": "ConnectError(\u0027All connection attempts failed\u0027)"
}
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
>>
Invoke-WebRequest:
{
  "detail": "ConnectError(\u0027All connection attempts failed\u0027)"
}
PS C:\Users\danie\azure-assignment> Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/generate?prompt=Azure_Prompt_v3"
>>

Statuscode      : 200
StatusDescription : OK
Content          : {"prompt":"Azure_Prompt_v3","text":"\"Azure_Prompt_v3\" is not an official, publicly documented Microsoft product or API version name. However, based on the pattern of
                  \\"Azure\\" (Microsoft's cloud pla...
RawContent       : HTTP/1.1 200 OK
                  Date: Thu, 04 Jun 2026 20:57:30 GMT
                  Server: unicorn
                  Set-Cookie: ARRAffinity=279e2686d6d985a92f8e554db67f8addb363f77898a222be3d5d0834f8dfedba;Path=/;HttpOnly;Secure;Domain=webapp-fas...
                  Headers: [{"Date", System.String[]}, {"Server", System.String[]}, {"Set-Cookie", System.String[]}, {"Content-Length", System.String[]}]
                  Images: {}
                  InputFields: {}
                  Links: {}
                  RawContentLength: 4729
                  Content: {"prompt":"Azure_Prompt_v3","text":"\"Azure_Prompt_v3\" is not an official, publicly documented Microsoft product or API version name. However, based on the pattern of
                  \\"Azure\\" (Microsoft's cloud pla...
                  RawContent: HTTP/1.1 200 OK
                  Date: Thu, 04 Jun 2026 20:57:30 GMT
                  Server: unicorn
                  Set-Cookie: ARRAffinity=279e2686d6d985a92f8e554db67f8addb363f77898a222be3d5d0834f8dfedba;Path=/;HttpOnly;Secure;Domain=webapp-fas...
                  Headers: [{"Date", System.String[]}, {"Server", System.String[]}, {"Set-Cookie", System.String[]}, {"Content-Length", System.String[]}]
                  Images: {}
                  InputFields: {}
                  Links: {}
                  RawContentLength: 4729
                  RelationLink: {}

PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/logs").Content
>>
{"logs":[{"Azure_Prompt_1","Azure_Prompt_2","2026-06-04 20:57:16,763 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:18,589 - ERROR - Error de comunicaci3n inesperado con servicio B: ConnectError('All connectio
attempts failed')","2026-06-04 20:57:24,707 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:24,877 - ERROR - Error de comunicaci3n inesperado con servicio B: ConnectError('All connection attempts failed')","2
26-06-04 20:57:30,817 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:46,528 - INFO - HTTP Request: GET http://backend:8888/generate?prompt=Azure_Prompt_v3 \\"HTTP/1.1 200 OK\\"", "2026-06-04 20:57:46,539 - INFO
- Respuesta exitosa recibida del servicio B."}]
PS C:\Users\danie\azure-assignment> az webapp restart --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP
>>
PS C:\Users\danie\azure-assignment> (Invoke-WebRequest "https://$WEB_APP_NAME.azurewebsites.net/logs").Content
>>
{"logs":[{"Azure_Prompt_1","Azure_Prompt_2","2026-06-04 20:57:16,763 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:18,589 - ERROR - Error de comunicaci3n inesperado con servicio B: ConnectError('All connectio
attempts failed')","2026-06-04 20:57:24,707 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:24,877 - ERROR - Error de comunicaci3n inesperado con servicio B: ConnectError('All connection attempts failed')","2
26-06-04 20:57:30,817 - INFO - Enviando prompt al servicio B en la URL http://backend:8888/generate","2026-06-04 20:57:46,528 - INFO - HTTP Request: GET http://backend:8888/generate?prompt=Azure_Prompt_v3 \\"HTTP/1.1 200 OK\\"", "2026-06-04 20:57:46,539 - INFO
- Respuesta exitosa recibida del servicio B."}]
PS C:\Users\danie\azure-assignment>

```

4.6. Control de Gastos y Limpieza de Recursos (Azure Cost Management)

Para evitar que los recursos aprovisionados sigan consumiendo saldo de tus créditos de estudiante o facturando importes adicionales tras haber finalizado la práctica, se aconseja gestionar la parada o eliminación del entorno con uno de los siguientes métodos:

Opción A: Detener la ejecución de la Web App (Conserva la configuración)

Si quieres pausar temporalmente el servicio (por ejemplo, para seguir haciendo pruebas en otro momento o para grabar el vídeo explicativo) deteniendo la ejecución de los contenedores:

```

# Detener la Web App
az webapp stop --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

# Reanudar la Web App
az webapp start --name $WEB_APP_NAME --resource-group $RESOURCE_GROUP

```

⚠ Nota: Ten en cuenta que si el Plan de App Service está en un tier de pago (como el SKU B1 recomendado para multi-contenedor), el coste del plan de hospedaje se seguirá cobrando por hora aunque la Web App esté detenida.

Opción B: Eliminar el Grupo de Recursos por completo (Recomendado al finalizar)

Si has completado toda la práctica y grabado el vídeo explicativo, la forma idónea de detener el gasto al 100% de manera definitiva es destruir el grupo de recursos completo. Esto eliminará de forma permanente la Web App, el App Service Plan, el Azure Container Registry (ACR) y la Storage Account:

```
# Eliminar el grupo de recursos completo sin esperar confirmación interactiva
az group delete --name $RESOURCE_GROUP --yes --no-wait
```

Referencias y Bibliografía

Para la elaboración de este despliegue y la configuración de los comandos utilizados a lo largo de la práctica, se han consultado las siguientes fuentes de documentación oficial:

Documentación de Azure CLI (Microsoft Learn):

- [az login](#) (Autenticación en Azure)
- [az group](#) (Administración de Grupos de Recursos)
- [az acr](#) (Gestión de Azure Container Registry)
- [az acr credential](#) (Obtención de Credenciales del Registro)
- [az appservice plan](#) (Configuración de Planes de App Service)
- [az webapp](#) (Creación y Gestión de Web Apps)
- [az webapp config container](#) (Configuración de Contenedores en Web Apps)
- [az webapp config appsettings](#) (Definición de Variables de Entorno y Configuración)
- [az webapp config storage-account](#) (Montaje de Almacenamiento en Web Apps)
- [az webapp log](#) (Descarga y Seguimiento de Logs en Tiempo Real)
- [az storage account](#) (Gestión de Cuentas de Almacenamiento)
- [az storage share-rm](#) (Gestión de Recursos Compartidos de Azure Files)

Documentación de Docker:

- [Referencia de la Línea de Comandos de Docker](#) (build, push, run, login)
- [Guía de Referencia del Archivo Dockerfile](#)
- [Docker Compose — Referencia de la CLI](#) (up, down, build, push)
- [Docker Compose — Especificación del Archivo docker-compose.yml](#)

PowerShell:

- [Invoke-WebRequest](#) (Cmdlet para peticiones HTTP)

Librerías y SDKs:

- [Documentación Oficial de FastAPI](#) (Framework Web)
- [SDK de Google GenAI para Python](#) (Gemini 2.5)
- [httpx](#) — Cliente HTTP asíncrono para Python

Anexo: Enlaces del Proyecto

A continuación se facilitan los enlaces correspondientes al repositorio de código fuente y a la demostración del despliegue:

- **Repositorio de GitHub:** [danielquinti/azure-assignment](#)

- **Prueba de Despliegue (Video):** [Demostración en vídeo de la práctica \(SharePoint\)](#)